

# AI for Economic Research: Dynamic Models, Language, and Agents

## Lecture 5.1: The RL Framework and Its Link to Dynamic Programming

● Zhigang Feng

2026-07-06 21:37:16

# Lecture 5 in One Map

One lecture, three decks, one idea: replace the model you must specify with experience you can sample — then scale it with function approximation.

you are here ▼

## 5.1 MODEL

MDP & the DP bridge

what is RL, and how does it  
relate to the DP I know?

FULL BACKUP (needs  $P$ ,  $R$ )

upstream: Lec02 (RL = 3rd ML pillar),  
Lec04 T2 (RL as a Bellman solver)

## 5.2 SAMPLE

MC · TD · Q-learning

how do I learn a policy with  
no model of the world?

SAMPLE BACKUP (model-free)

## 5.3 SCALE

actor-critic · deep RL

how does it scale — and  
where has it worked?

SAMPLED + APPROXIMATED

downstream: Lec06 — HA models at  
scale (Aiyagari, Krusell-Smith, DeepHAM)

**Act 5.1 (here)** opens with intuition and everyday examples, then builds the formal toolkit: the MDP framing, the Bellman equation economists already use, and the exact point where classical DP breaks — the wall the next two acts climb. Running anchor throughout: the optimal growth model.

# What Is Reinforcement Learning?

Intuition and everyday examples, before any formalism

# Reinforcement Learning: Learning by Trial and Error

- **Reinforcement learning (RL)**: an **agent** learns to make good decisions by **interacting** with an **environment** and observing a **reward**.
- No teacher hands it the right answer — it learns from **consequences**: try, observe, adjust, repeat.

## How RL differs from the other ML pillars:

- **Supervised** learning: data come with *labels* (the correct answer).
- **Unsupervised** learning: find *structure* in unlabeled data.
- **Reinforcement** learning: no labels — only a *scalar reward*, often *delayed*, to be maximized over time.

# What Is RL? Three Everyday Examples

| Setting                              | Action                | Reward                | What it learns          |
|--------------------------------------|-----------------------|-----------------------|-------------------------|
| Playing a game<br>(Go, Atari, chess) | a move                | win / lose, score     | a winning <b>policy</b> |
| A child / robot<br>learning to walk  | motor commands        | progress; didn't fall | a stable <b>gait</b>    |
| A/B testing<br>(web page, price)     | which version to show | click / purchase      | the best <b>option</b>  |

**Common thread:** decisions are **sequential**, rewards can be **delayed**, and the agent must **explore** (try things) — not just exploit what looks best so far.

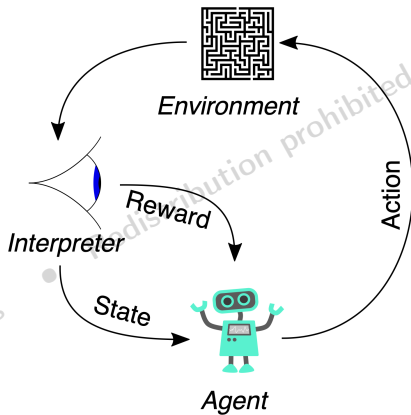
# The Same Loop in Economics

- **Dynamic pricing:** a firm sets a *price* (action), observes *demand* / *profit* (reward), learns a **pricing rule**.
- **Monetary policy:** a central bank sets the *interest rate* (action), observes *inflation* / *output* (reward), learns a **policy rule**.
- **Consumption–savings:** a household chooses *how much to save* (action), enjoys *utility now* and carries *wealth forward* (state), learns a **savings rule**.

These are exactly the **sequential decision problems** economists have long solved with **dynamic programming**. RL is a different way to attack the *same* problems — by learning from experience.

# The Common Pattern: Agent, Environment, Reward

At each time step, the agent observes state  $S_t$ , chooses action  $A_t$ . The environment returns reward  $R_{t+1}$  and transitions to  $S_{t+1}$ .



**Trajectory:**  $s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots$  The **return** (cumulative reward) from time  $t$ :  $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ .

# Two Ways to Solve a Dynamic Problem: Map vs. Explore

## Dynamic Programming

*“model first, then solve”*

- Specify the environment fully ( $P, R$ ).
- Compute  $V$  at **every** state — draw the complete map.
- Then act by the map.
- Like an **omniscient planner**.

## Reinforcement Learning

*“interact and learn”*

- No complete map.
- **Walk** the environment repeatedly; update route judgments after each trip.
- Estimates approach what the map would say — without ever drawing it.
- Like an **explorer**.

Same destination (the optimal policy), two routes. RL shines exactly when drawing the full map is **infeasible** — the theme of this lecture.



# Why RL Exists: High Dimensions, Opaque Environments — and Deep Learning

The engineering problem RL was built for:

- **Scale.** The state space can be astronomical *even when the rules are known*: Go has  $\approx 10^{170}$  legal positions (vs.  $\sim 10^{80}$  atoms in the universe), branching  $\sim 250$  per move. No table; no full enumeration.
- **Opacity.** Robot dynamics, traffic, an opponent — often **cannot be written down**, only **interacted with**.
- **Evaluative, delayed feedback.** You get a *reward* (win/lose, score), not the correct action  $\Rightarrow$  the **credit-assignment problem** (Minsky): which past decisions earned the outcome?

The core RL ideas date to the 1980s–90s, but RL only “flew” once **deep learning** supplied efficient **function approximation in high dimensions**: **DQN** (Atari from raw pixels, 2015) and **AlphaGo** (2016). We return to AlphaGo in T3.

# The RL Framework

MDPs, returns, policies, and value functions — the formal toolkit

# From Static to Dynamic: Why RL?

- Previous lectures covered **static** tasks: prediction, classification, curve fitting.
- Many core economic problems are **sequential**:
  - Consumption–saving decisions
  - Firm investment and pricing
  - Monetary and fiscal policy
- Three features distinguish dynamic problems:
  1. **Sequential**: decisions at every period, not once-and-for-all.
  2. **Intertemporal coupling**: today's consumption  $\Rightarrow$  tomorrow's capital.
  3. **Uncertainty**: future shocks  $z_{t+1}$  unknown at time  $t$ .
- Economists traditionally use **Dynamic Programming** (DP) to solve these.
- **Reinforcement Learning** extends DP to handle high dimensions and unknown environments.

# Reinforcement Learning in Economics

- RL is designed for **sequential decision-making under uncertainty**.
- Agents maximize long-term objectives: utility, profit, welfare.
- RL differs from supervised/unsupervised learning by **interacting with an environment** to learn optimal policies.
- Natural fit for economic models involving dynamic optimization.

*(Recall the map-vs-explore contrast from the intro: DP models everything first; RL interacts and learns.)*

*Closing the loop:* Lec02 T1 introduced RL as one of the three ML pillars; Lec04 T2 previewed RL as a Bellman solver. Lec05 formalizes the MDP framing, classical-DP limits, and the algorithmic family that turns Lec04's preview into a publication-grade toolkit.

# Markov Decision Process (MDP) Formulation

- An RL problem is formalized as a **Markov Decision Process (MDP)**:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle.$$

- $\mathcal{S}$ : set of **states** (e.g., capital  $k$ , shock  $z$ , wealth distribution  $\Phi$ ).
- $\mathcal{A}$ : set of **actions** (e.g., consumption  $c$ , investment  $i$ ).
- $P : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ ,  $P(s'|s, a)$ : **transition kernel**.
- $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ ,  $R(s, a)$ : **reward** (= instantaneous utility  $u(c)$ ). *Canonical form used throughout Lec05–Lec09.*
- $\gamma \in [0, 1)$ : **discount factor** (=  $\beta$  in econ notation).
- **Objective**: Find an optimal policy  $\pi^*$  that maximizes

$$\mathbb{E}_{\pi} \left[ \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right].$$

# MDP Meets the Growth Model

Consider the **optimal growth model**:

$$\max_{\{c_t, k_{t+1}\}} \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \quad \text{s.t.} \quad c_t + k_{t+1} = z_t f(k_t) + (1 - \delta)k_t$$

| MDP element             | Growth model                                                                                        |
|-------------------------|-----------------------------------------------------------------------------------------------------|
| State $s$               | $(k_t, z_t)$ — capital + productivity                                                               |
| Action $a$              | $c_t$ (or equivalently $k_{t+1}$ )                                                                  |
| Transition $P(s' s, a)$ | $k_{t+1} = (1 - \delta)k_t + z_t f(k_t) - c_t$ ; $\log z_{t+1} = \rho \log z_t + \varepsilon_{t+1}$ |
| Reward $R(s, a)$        | $u(c_t)$ — period utility                                                                           |
| Discount $\gamma$       | $\beta$ — household discount factor                                                                 |
| Policy $\pi(s)$         | Consumption rule $c^*(k, z)$                                                                        |

The Bellman equation and MDP are two views of the **same** problem.

# Policy and Value Functions

- **Policy**  $\pi$ : a mapping from states to actions.
  - Deterministic:  $a = \pi(s)$  (e.g.,  $c^*(k, z)$  in econ).
  - Stochastic:  $a \sim \pi(a|s)$  (useful for exploration).
- **State-value function** — expected return from state  $s$  under  $\pi$ :

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] = \mathbb{E}_\pi \left[ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right].$$

- **Action-value function** — expected return for action  $a$  in state  $s$ :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].$$

- **Relationship:**  $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$ .

## Connection to Dynamic Programming

The Bellman equation — economists' old friend — is RL's backbone



# Why Economists Specify, Then Solve

- Economists **write the model down first** — preferences  $u(\cdot)$ , technology  $f(k)$ , constraints, shock process — *then* solve it.
- **Why?** Each primitive carries economic meaning ( $\beta$  patience,  $\alpha$  factor share,  $\delta$  depreciation), so the solution inherits:
  - **Interpretability** — parameters are economic magnitudes.
  - **Welfare analysis** — integrate the same  $u(c)$  that defines behavior.
  - **Equilibrium-consistent prices** —  $r = zf'(k) - \delta$ ,  $w = z[f(k) - kf'(k)]$  are *competitive factor prices*, not reduced-form fits.
- Disciplined by the **Lucas critique**: decision rules *shift* when policy shifts, so a rule fit to historical behavior can fail once deployed — better to solve a **structural** model.

**Honest caveat:** the workhorse is usually **calibrated**, not structurally estimated — so interpretability is *necessary, not sufficient*, for identification. (Revisited in T3: *RL scales the solution, not the identification.*)

# The Bellman Equation — Economists' Old Friend

- Economists compute the value function  $V(s)$  via the Bellman equation:

$$V(s) = \max_a \left\{ R(s, a) + \gamma \mathbb{E}[V(s') \mid s, a] \right\}.$$

- Define the **Bellman operator**  $T$ :

$$(TV)(s) = \max_a \left\{ R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V(s') \right\}.$$

- The value function is the **fixed point**:  $V^* = TV^*$ .

# Theoretical Foundations of DP

Three pillars guarantee that DP “works”:

## 1. Principle of Optimality (Bellman):

- Any tail of an optimal plan is itself optimal from the state it reaches.
- Guarantees **equivalence** between the sequence problem and the recursive formulation.

## 2. Contraction Mapping Theorem (Banach fixed-point theorem):

- $\|TV_1 - TV_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$  for any  $V_1, V_2$ .
- Guarantees **existence, uniqueness, and convergence** of VFI.

## 3. Blackwell's Sufficient Conditions

Blackwell (1965), *Ann. Math. Stat.*:

- **Monotonicity:**  $V_1 \leq V_2 \Rightarrow TV_1 \leq TV_2$ .
- **Discounting:**  $T(V + c) \leq TV + \gamma c$  for  $c \geq 0$ .
- Easy-to-check conditions that imply contraction.

*Economists prize uniqueness: it makes “the model predicts  $X$ ” well-posed — no ambiguity over which solution to report, which is what lends structural counterfactuals their credibility.*

## Full Expectation Backup — DP's Core Computation

The heart of Value Function Iteration (VFI):

$$(TV)(k, z) = \max_{k'} \left\{ u(c) + \beta \sum_{z' \in \mathcal{Z}} P(z'|z) V(k', z') \right\}$$

**Two prerequisites:**

1. Must know the **law of motion**: how  $(k', z')$  is determined by  $(k, z, a)$ .
2. Must know the **transition probabilities**  $P(z'|z)$ .

This is a **full expectation backup**: to update  $V$  at one state, sum over **all possible next states** weighted by their probabilities.

If either the law of motion or the transition probabilities are unknown (or too expensive to compute), DP cannot proceed.  $\Rightarrow$  **RL replaces full expectation backup with sample backup.**

# $V(s)$ vs. $Q(s, a)$ : Economics vs. Engineering

## Economics: Focus on $V(s)$

- Emphasis on **welfare/utility** from a given state.
- Optimal policies derived from planner's problem or equilibrium.
- DP solves for  $V^\pi(s)$  directly.
- Rational agents optimize implicitly.

## RL / Engineering: Focus on $Q(s, a)$

- Emphasis on **real-time action selection**.
- Learning  $Q(s, a)$  allows decision-making **without a model**.
- $\pi^*(s) = \arg \max_a Q^*(s, a)$ .
- Works in unknown environments.

**Growing convergence:** As economists tackle complex, partially-known environments (heterogeneous agents, financial markets), the Q-function approach becomes increasingly valuable.

## Limitations of Classical DP

Two walls — high dimensions and unknown environments — and why RL helps

## Challenge 1: The Curse of Dimensionality

- VFI requires a **grid** over the state space.
- 2D model  $(k, z)$ :  $100 \times 100 = 10,000$  points — easy.
- Multi-sector model with  $d$  state variables,  $m$  grid points each:  $m^d$  points.
  - $d = 5$ ,  $m = 100$ :  $10^{10}$  points.
  - $d = 8$ ,  $m = 100$ :  $10^{16}$  points — **completely infeasible**.
- Sparse grids (Smolyak) and projection methods (Chebyshev) partially mitigate this, but break down beyond  $\sim 5$ – $8$  dimensions.

**Heterogeneous agent models** make it worse:

- The wealth **distribution**  $\Phi$  is itself a state variable — an infinite-dimensional object.
- Krusell–Smith (1998) approximates  $\Phi$  by its mean  $K$ , but accuracy degrades in complex models.

## Challenge 2: Unknown or Intractable Environments

- In a representative-agent model: agent knows own state  $\Rightarrow$  knows prices  $\Rightarrow$  environment is **transparent**.
- In a heterogeneous-agent model:
  - Prices depend on aggregate capital  $K = \int a d\Phi(a, z)$ .
  - Individual agent does **not observe**  $\Phi$  directly.
  - The distribution's law of motion  $\Phi' = \mathcal{T}(\Phi)$  has no closed form in general.
- The “environment” (price dynamics) that an agent faces is **endogenous** and **unknown**.

**The core issue for economists:** prices are the environment. When prices depend on distributions you cannot track, the environment becomes something you must *learn* rather than *specify*.



# Why (Deep) Reinforcement Learning Helps

## 1. Flexible Function Approximation (Neural Networks):

- Traditional DP uses structured grids (Chebyshev, splines) that scale exponentially.
- Neural networks approximate  $V(s)$  or  $\pi(s)$  **without a grid**; parameters scale gracefully with dimension.

## 2. Sample-Based Learning (No Full Expectation Backup):

- RL uses **simulated trajectories** instead of summing over all  $s'$ .
- Targets **relevant regions** of the state space via experience.
- Especially valuable when  $\Phi$ 's law of motion lacks a closed form.

## 3. Hardware Acceleration (GPUs/TPUs):

- NN training = large-scale matrix multiplication  $\Rightarrow$  GPU-friendly.
- Traditional grid-based DP does not map as naturally onto GPU architectures.

## When DP, When RL? A $2 \times 2$ Map

|          | Model known ( $P$ available)                   | Model unknown / intractable                    |
|----------|------------------------------------------------|------------------------------------------------|
| Low-dim  | Textbook DP (VFI / PI) — <i>RL buys little</i> | Tabular RL (Q-learning, SARSA)                 |
| High-dim | “Deep DP”: NN approximation <i>inside</i> VFI  | Deep RL’s home turf (HA models, market design) |

**DP is RL’s structural special case:** when  $P$  is known and  $|\mathcal{S}|$  is small, sample backup  $\rightarrow$  full backup and function approximation  $\rightarrow$  table. (A statement about the *equations*, not a free convergence guarantee.) So RL **extends** DP — it does not replace it. And RL is *not merely a last resort*: even with  $P$  known, sampling + approximation can win in high dimensions.

## RL: Beyond Classical DP — Summary

- RL algorithms (Q-learning, SARSA, actor-critic) extend DP to:
  - Handle **unknown** transition models (model-free).
  - Address **exploration–exploitation** directly.
  - **Scale up** through function approximation.
- Two pathways:
  - **Model-free RL**: learn policies purely from observed data without explicit transition models.
  - **Model-based RL**: combine simulation from approximate structural models with learning.

The two RL pathways differ only in *where the samples come from* — a real/observed environment (model-free) or a simulated structural model (model-based). Either way, the Bellman backbone is unchanged; only the *backup* changes.

## Bridge: From a Model You Specify to Experience You Sample

Act 5.1 framed the problem. Every sequential economic decision is an MDP, and the Bellman equation economists already use is RL's backbone — but the full-expectation backup it relies on needs a known model  $(P, R)$  and a small enough state space to sweep.

- You have seen *what* RL is, *why* it coincides with classical DP, and *where* DP hits a wall: the curse of dimensionality and unknown / endogenous environments (prices you cannot track).
- **Next (5.2):** the question changes from “how do we *specify* the model?” to “how do we *learn* a good policy *without one?*” — Monte Carlo, temporal-difference, and Q-learning replace the full expectation backup with a **sample** backup.

Arc: **5.1 MODEL** (framework + DP)  $\rightarrow$  **5.2 SAMPLE** (MC / TD / Q-learning)  $\rightarrow$  **5.3 SCALE** (actor-critic + deep RL + applications).